Vibe code is legacy code
氛围代码已成遗留代码

Blog.val.town Val Town 博客

Despite widespread confusion, Andrej Karpathy coined "vibe coding" as a kind of AI-assisted coding where you **"forget that the code even exists.”**

尽管存在普遍误解，但安德烈·卡帕西提出的"氛围编程"本质上是一种 AI 辅助编程方式——让你"完全忘记代码本身的存在"。

Legacy code 祖传代码

We already have a phrase for code that nobody understands: **legacy code**.
我们早就有个术语专指无人能懂的代码：祖传代码。

Legacy code is universally despised, and for good reason. But why? You have the code, right? Can't you figure it out from there?
祖传代码遭到全行业的深恶痛绝，这绝非没有道理。但为什么呢？代码明明就在那里，难道不能通过阅读来理解吗？

Wrong. Code that nobody understands is tech debt. It takes a lot of time to understand unfamiliar code enough to debug it, let alone introduce new features without also introducing bugs.
错。无人理解的代码就是技术债务。要花大量时间才能充分理解陌生代码以进行调试，更不用说在不引入新错误的情况下添加新功能了。

Programming is fundamentally *theory building*, not producing lines of code. We know this. This is why we make fun of business people who try to measure developer productivity in lines of code.

编程本质上是理论构建，而非生产代码行数。我们心知肚明。正因如此，我们才会嘲笑那些试图用代码行数衡量开发效率的商业人士。

When you vibe code, you are incurring tech debt as fast as the LLM can spit it out. Which is why vibe coding is *perfect* for prototypes and throwaway projects: It's only legacy code if you have to maintain it!
当你凭感觉编码时，你积累技术债务的速度就和 LLM 生成代码的速度一样快。这也正是感觉编码特别适合原型和一次性项目的原因：只有需要维护的代码才会变成遗留代码！

Prototypes & throwaway code
原型与一次性代码

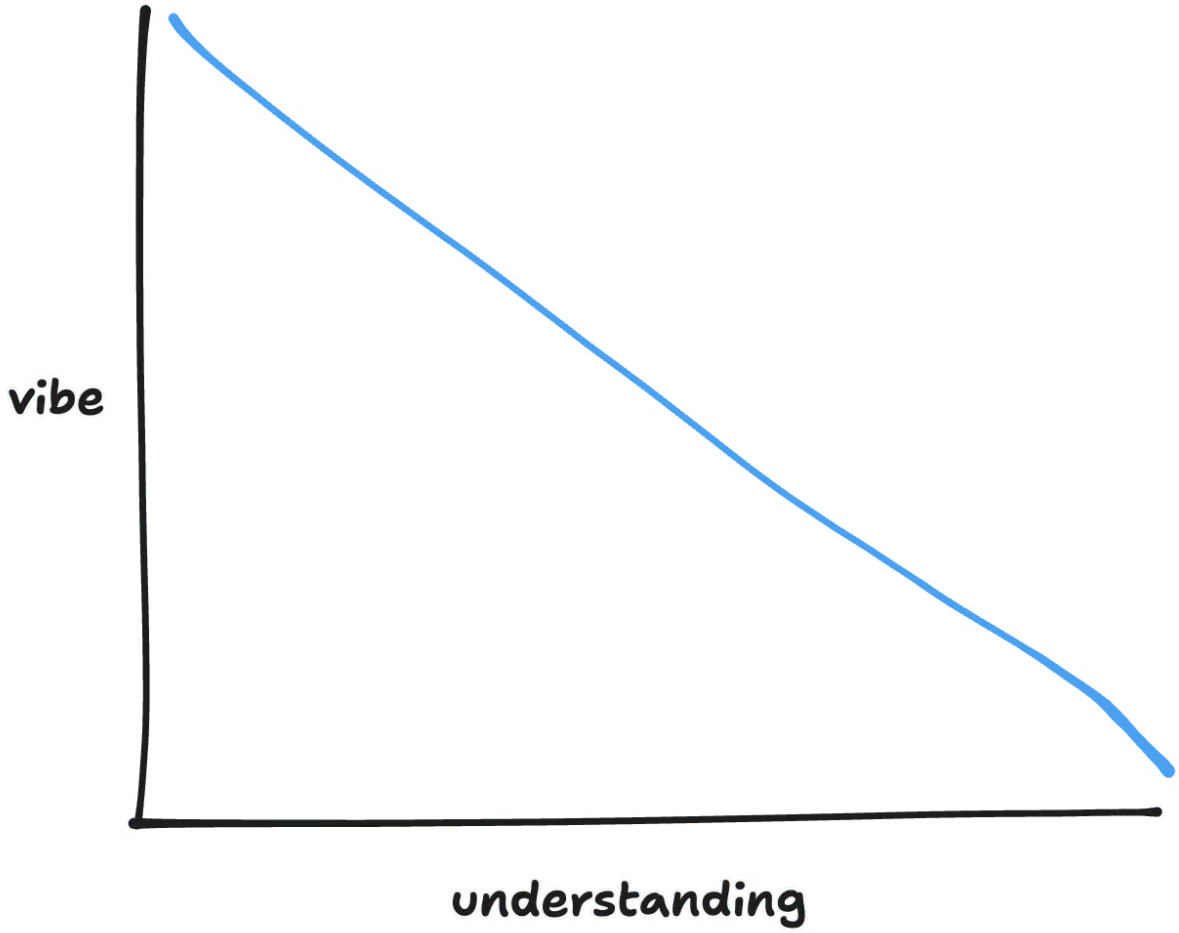
I've happily vibe coded apps to:
我曾愉快地用氛围编码开发过以下应用：

- Calculate weekly growth rates
  计算周增长率
- Rate NYT Connections 给《纽约时报》Connections 游戏评分
- Propose to my fiance
  向我的未婚妻求婚

I don't need to continue developing those apps, so it hasn't been a problem that I don't understand their code. These apps are also very small, which means that I haven't incurred that much debt if I need to jump in and read the code at some point. I was able to vibe code these apps way faster than I could've built them, and it was a blast.
我不需要继续开发那些应用，所以不理解它们的代码并不是问题。这些应用也非常小，这意味着即使某天我需要介入阅读代码，也不会背负太多技术债务。我能够通过直觉编码比常规开发更快地完成这些应用，这过程非常有趣。

Vibe coding is a spectrum
直觉编码是一个连续谱系

Vibe coding is on a spectrum of how much you understand the code. The more you understand, the less you are vibing.
直觉编码取决于你对代码的理解程度。理解得越多，直觉编码的成分就越少。



Simply by being an engineer and asking for a web app with a persistent database, you are already vibing less than than a non-programmer who asks for an "app" without understanding the distinction between a web app and a native app, or how persistent data storage works.
当你作为工程师要求开发一个带持久化数据库的网页应用时，相比那些分不清网页应用和原生应用、也不懂数据持久化原理的非程序员单纯要求做个"应用"，你已经在减少直觉编码的成分了。

Giving a credit card to a child
把信用卡交给小孩

The worst possible situation is to have a non-programmer vibe code a large project that they intend to maintain. This would be the equivalent of giving a credit card to a child without first explaining the concept of debt.
最糟糕的情况莫过于让一个非程序员凭感觉编写一个他们打算长期维护的大型项目。这就像在没解释债务概念之前就把信用卡交给孩子一样危险。

As you can imagine, the first phase is ecstatic. *I can wave this little piece of plastic in stores and take whatever I want!*
可想而知，第一阶段是狂喜的。我可以在商店里挥动这张小塑料片，拿走任何想要的东西！

Which is a lot like *AI can build anything now! Nobody needs to learn how to code! Look at what it just made for me!*
这很像现在 AI 无所不能！谁还需要学编程！看看它刚给我做的东西！

But if you wait a month, you'll get the credit card bill. *Did I actually need to buy all those things? How will I get myself out of this hole?*
但如果你等上一个月，就会收到信用卡账单。我真的需要买那些东西吗？我该如何摆脱这个困境？

It's similar for the vibe coder. *My code is broken. What do all these files and folders even do? How will I ever get this fixed? Can I get a refund for the $400 I spent vibe coding?*
对于氛围编程者来说也是如此。我的代码一团糟。这些文件和文件夹到底是干什么用的？我该怎么修复这个问题？我能要回为氛围编程花掉的 400 美元吗？

If you don't understand the code, your only recourse is to ask AI to fix it for you, which is like paying off credit card debt with another credit card.
如果你看不懂代码，唯一的办法就是让 AI 帮你修复，这就像用另一张信用卡偿还信用卡债务。

Serious coding with AI in 2025
2025 年用 AI 进行严肃编程

If you're building something serious that you intend to maintain in 2025, Andrej has the right of it:
如果你正在构建一个打算在 2025 年仍要维护的重要项目，Andrej 的观点是正确的：

[Keep] a very tight leash on this new over-eager junior intern savant with encyclopedic knowledge of software, but who also bullshits you all the time, has an over-abundance of courage and shows little to no taste for good code. And emphasis on being slow, defensive, careful, paranoid, and on always taking the inline learning opportunity, not delegating.

[务必]严加管束这位热情过度的新晋实习生天才，他虽拥有百科全书般的软件知识储备，却总对你信口开河，勇气过剩而对优秀代码几乎毫无鉴赏力。要特别强调慢工出细活、防御性思维、谨小慎微、多疑态度，并始终把握嵌入式学习机会而非委派他人。

— Andrej Karpathy, twitter
——安德烈·卡帕西（Andrej Karpathy），推特

How we approach building for AI
我们如何构建 AI 产品

At Val Town, we've built AI into our product in dozens of ways. Townie is our AI assistant that agentically reads & writes code, runs it, views the logs, and keeps iterating until it's done.
在 Val Town，我们以数十种方式将 AI 融入产品。Townie 是我们的 AI 助手，它能自主阅读和编写代码、运行程序、查看日志，并持续迭代直至完成任务。

Townie is an awesome tool for vibe coding. I heartily recommend it to folks who understand these tradeoffs. I use it to vibe code sometimes. Other times I keep it on a tight leash as it makes surgical edits to a project I care about. Both are fun and useful.

Townie 是进行氛围编程的绝佳工具。我由衷地向理解这些权衡取舍的朋友推荐它。有时我会用它来随心编程，其他时候则会严格控制使用——比如在对重要项目进行精细修改时。两种方式都充满乐趣且实用。

Coding with AI is changing so quickly that it's hard to know what tomorrow will bring, but I'm confident that theory building will remain central to the activity of building complex software. Our technical expertise will still be relevant! And I'm optimistic that AI will continue to make programming better in surprising ways.
与 AI 共同编程的发展日新月异，很难预知明天会带来什么变化，但我确信理论构建仍将是开发复杂软件的核心活动。我们的技术专长依然不可或缺！我乐观地认为，AI 将继续以出人意料的方式让编程变得更好。

But if you know any non-programmers spending thousands of dollars vibe coding their billion dollar app idea today, please send them this post. Vibe coding is not going to get them where they want to go. They're going to have to learn to use their human eyes to read the code 😱, and learn that sometimes it's easier to start over with building a well-written code base from scratch than to fix a legacy one that nobody understands.
但如果你认识任何非程序员今天正花费数千美元用氛围编码来实现他们价值十亿美元的应用程序创意，请把这篇文章发给他们。氛围编码不会带他们到达想去的地方。他们必须学会用人类的双眼去阅读代码😱，并明白有时从头开始构建一个编写良好的代码库，比修复一个无人理解的遗留系统要容易得多。